ADS205 Tutorial 1 - Strings, Languages and Proofs

September 23, 2026

1 ADS205 · Decision, Computation and Language

2 Tutorial 1 — Strings, Languages, and Writing Proofs

Solutions are folded beneath each problem — click the arrow to expand them. Attempt the problem first.

2.1 Reference card

Object	Definition
Alphabet Σ	a nonempty **finite** set of symbols, e.g. $\{\texttt{0}, \texttt{1}\}$
Symbol	an element of Σ, written in typewriter font: $\texttt{a}$
String	a **finite** sequence of symbols from Σ
Σ^*	the set of **all** strings over Σ
ε	the empty string, the string of length 0
$\lvert w \rvert$	length of w; w_i is the i-th symbol of w
xy (or $x \circ y$)	concatenation; $w^2 = ww$, $w^3 = www$
Prefix / suffix	x is a prefix of w if $w = xy$ for some y; then y is a suffix
Substring	y is a substring of w if $w = xyz$ for some x, z
Language	a **set** of strings over Σ

Conventions used throughout this module:

- $\texttt{a}$ is a symbol; a is a *variable* ranging over symbols.
- We do not distinguish a symbol from the length-1 string containing it.
- ε is **not** a symbol: $\varepsilon \notin \Sigma$, but $\varepsilon \in \Sigma^*$.
- $w\varepsilon = \varepsilon w = w$, and concatenation is associative, so we write xyz without brackets.
- $\emptyset$ is the language with **no** strings; $\{\varepsilon\}$ is the language with **one** string, of length 0.

Operations on languages. For languages A, B, with w^R the reversal of a string:

$$A \cup B = \{w \mid w \in A \text{ or } w \in B\} \qquad A \circ B = AB = \{xy \mid x \in A,\ y \in B\} \qquad L^R = \{w^R \mid w \in L\}$$

$$L^* = \{w_1 w_2 \cdots w_k \mid k \geq 0 \text{ and } w_i \in L \text{ for } i = 1, \ldots, k\}$$

Note $k \geq 0$: $k = 0$ **puts** ε **into every** L^*, for every language L, without exception.

3 Q & A

1. Does a finite automaton need to have at least one state?
2. What if we read a character but there is no corresponding transition?
3. Can we read a character but conduct no transition?
4. Can the empty string be accepted by a finite automata?
5. Is an empty language regular?

Solution

1. Yes, the start state. Every finite automaton must have a start state.
2. This is possible only in certain finite automata
3. No, the machine either transitions to a new state or directly reject the string
4. Yes, when the start state is the accept state.
5. Yes, its corresponding finite automaton has no accept state.

---

4 Part A — Strings and notation

4.1 Problem 1 · True or false?

Assume $\Sigma = \{\texttt{a}, \texttt{b}, \texttt{c}\}$.

On symbols and strings:

1. $\Sigma \subseteq \Sigma^*$
2. $\varepsilon \in \Sigma$
3. $\varepsilon \in \Sigma^*$
4. $\texttt{a} = \texttt{aa}$
5. $\texttt{ab} = \texttt{ba}$

On languages:

6. $\varepsilon \in \{\texttt{abc}\}$
7. $\{\texttt{abc}\} \cup \emptyset = \{\texttt{abc}\}$
8. $\{\texttt{abc}\} \cup \{\varepsilon\} = \{\texttt{abc}\}$
9. $\Sigma^* = \Sigma^* \cup \{\varepsilon\}$

Solution

1. **True.** We do not distinguish a symbol from the length-1 string containing it, so every element of Σ is also an element of Σ^*.
2. **False.** Σ is a set of *symbols*; ε is a string of length 0, not a symbol. (The single most common Week-1 mistake.)
3. **True.** Σ^* is the set of all *finite* sequences of symbols, and the length-0 sequence is finite.
4. **False.** Sequences are equal only if they have the same length and matching symbols: $|\texttt{a}| = 1 \neq 2 = |\texttt{aa}|$.
5. **False.** Order matters: position 1 holds `a` in one string and `b` in the other.
6. **False.** $\{\texttt{abc}\}$ contains exactly one string, `abc`, and $\texttt{abc} \neq \varepsilon$.

7. **True.** $\emptyset$ contributes no elements, so the union is unchanged.
8. **False.** $\{\texttt{abc}\} \cup \{\varepsilon\} = \{\texttt{abc}, \varepsilon\}$: two strings, hence a different set.
9. **True.** $\varepsilon \in \Sigma^*$ already, so adding it changes nothing. (Contrast with 8: whether ε is absorbed depends on whether it was already there.)

4.2 Problem 2 · Prefixes, length, substrings

1. List **all** the prefixes of $\texttt{abc}$.
2. What is $|\varepsilon\varepsilon\varepsilon|$?
3. Give an example where xz is **not** a substring of xyz. Then give an example where it **is**.

Solution

1. ε, $\texttt{a}$, $\texttt{ab}$, $\texttt{abc}$ — four of them. (Forgetting ε and the whole string are the two usual omissions.)
2. 0. Concatenating empty strings gives the empty string: $\varepsilon\varepsilon\varepsilon = \varepsilon$, and $|\varepsilon| = 0$.
3. **Not**: $x = \texttt{a}, y = \texttt{b}, z = \texttt{c}$, so $xyz = \texttt{abc}$ and $xz = \texttt{ac}$, which does not occur contiguously. **Is**: take $y = \varepsilon$. Then $xyz = xz$, and every string is a substring of itself. (Also $x = y = z = \texttt{a}$ works: $\texttt{aaa}$ contains $\texttt{aa}$.)

4.3 Problem 3 · What is allowed to be infinite?

According to the definitions above, does there exist

1. an infinite **alphabet**?
2. an infinite **string**?
3. an infinite **language**?

Answer yes/no and justify from the definition, not from intuition.

Solution

1. **No.** An alphabet is defined as a *finite* set of symbols.
2. **No.** A string is defined as a *finite* sequence.
3. **Yes.** A language is just a set of strings, and a set of finite things can be infinite: $\Sigma^* = \{\varepsilon, \texttt{a}, \texttt{b}, \texttt{aa}, ...\}$ is infinite, and so is $PRIME = \{\texttt{2}, \texttt{3}, \texttt{5}, \texttt{7}, \texttt{11}, ...\}$.

The asymmetry is deliberate, and it is what makes this module possible: **inputs are always finite, but the problems we pose range over infinitely many inputs.** "Is this string prime?" is one question about infinitely many possible inputs. Problem 8 pushes on exactly this point.

---

5 Part B — Operations on languages

5.1 Problem 4 · The two languages that break intuition

Compute, straight from the definitions:

1. $\emptyset \circ \{\texttt{a}\}$
2. $\{\varepsilon\} \circ \{\texttt{a}\}$
3. $\emptyset^*$
4. $\{\varepsilon\}^*$

Then answer: is $L \circ \emptyset = \emptyset$ true for **every** language L? Is $L^* = \emptyset$ possible for some L?

Solution

1. $\emptyset$. By definition $A \circ B = \{xy \mid x \in A, y \in B\}$; there is no $x \in \emptyset$, so no pair (x, y) exists and the result is empty.
2. $\{\mathtt{a}\}$. The only pair is $(\varepsilon, \mathtt{a})$ and $\varepsilon\mathtt{a} = \mathtt{a}$.
3. $\{\varepsilon\}$. In the definition of L^* the case $k = 0$ is always available and contributes ε; with $L = \emptyset$ no $k \geq 1$ contributes anything. So $\emptyset^* = \{\varepsilon\} \neq \emptyset$.
4. $\{\varepsilon\}$. Every product $\varepsilon\varepsilon \cdots \varepsilon = \varepsilon$, so the set is $\{\varepsilon\}$ however many factors you take.

$L \circ \emptyset = \emptyset$ for every L: **yes**, for the same reason as (1) — the second component can never be supplied. $L^* = \emptyset$: **impossible**, since $\varepsilon \in L^*$ for every L, including $L = \emptyset$.

Rule of thumb: $\emptyset$ annihilates concatenation, but the star always leaves ε behind.

5.2 Problem 5 · Compute with concrete languages

Let $A = \{\mathtt{c}, \mathtt{d}\}$, $B = \{\mathtt{at}, \mathtt{og}\}$ and $L = \{\mathtt{ac}, \mathtt{bd}\}$.

1. Compute $A \circ B$.
2. List every string of L^* whose length is ≤ 4, and say which value of k produces each one.
3. Is $\mathtt{a} \in L^*$? Is $\mathtt{aaaa} \in L^*$?

Solution

1. $A \circ B = \{\mathtt{cat}, \mathtt{cog}, \mathtt{dat}, \mathtt{dog}\}$ — all four pairings; it does not matter that not all of them are English words.
2. $k = 0$: ε (length 0). $k = 1$: $\mathtt{ac}$, $\mathtt{bd}$ (length 2). $k = 2$: $\mathtt{acac}$, $\mathtt{acbd}$, $\mathtt{bdac}$, $\mathtt{bdbd}$ (length 4).
3. $\mathtt{a} \notin L^*$. $\mathtt{aaaa} \notin L^*$.

---

6 Part C — Writing proofs

Read this before starting Problem 6. Writing a proof is much closer to writing a program than it looks.

Before you write. 1. *Understand the requirements.* What exactly is given (premises, definitions, facts you may use) and what exactly must be concluded? Rewrite the statement more explicitly if that helps. 2. *Develop an intuition.* Try examples — and try to pick examples that would **falsify** the claim if it were false. Choosing good examples is choosing good test cases. 3. *Divide and conquer.* Split the goal into smaller claims. "Prove $X = Y$" almost always becomes "prove $X \subseteq Y$" and "prove $X \supseteq Y$".

Two formats. A *statement–reason* proof lists numbered statements, each with the reason it follows from earlier ones. It makes every step explicit but would run to hundreds of pages. In practice we write *paragraph proofs*: English that convinces the reader a statement–reason version **could** be written. Both appear below, and translating between them is the best exercise there is for learning where the real steps are.

Paragraph proofs. Start from the givens and reason forwards to the conclusion; never start from the conclusion and work backwards, and never assume what you are proving. It is fine — good,

even — to insert a short "comment" saying where a stretch of proof is heading ("To show…", "It suffices to show…", "Claim:").

Generality. To prove "there exists X" one example suffices; to *disprove* "for all X" one counterexample suffices. But to prove "for all X" over an infinite collection, no number of examples is a proof. Examples may appear inside a proof to clarify a step, never to replace the general argument.

Detail. Enough detail that a classmate could reconstruct the missing steps, and that every step specific to *this* module (strings, languages, automata) is explicit. Standard set-theoretic moves ("$w \in A \cup B$, so $w \in A$ or $w \in B$") need no comment.

6.1 Problem 6 · Practice: converting between the two formats

(a) Convert this paragraph proof into a statement–reason proof. Be sure to write **which statement(s) each statement depends on**.

> **To show:** If s is a string, every substring of a substring of s is a substring of s.
>
> **Proof:** Let y be a substring of s, that is, $s = xyz$ for some x, z; and let v be a substring of y, that is, $y = uvw$ for some u, w. Then $s = xuvwz$, so v is a substring of s. □

(b) Convert this statement–reason proof into a paragraph proof.

> **To show:** If w is a string, every prefix of a suffix of w is a suffix of a prefix of w.

#	Statement	Reason
1	v is a suffix of w	Given
2	y is a prefix of v	Given
3	$\exists x$ s.t. $xv = w$	(1), def. suffix
4	$\exists z$ s.t. $yz = v$	(2), def. prefix
5	$xyz = w$	(3), (4), substitution
6	xy is a prefix of w	(5), def. prefix
7	y is a suffix of xy	(6), def. suffix

Solution

(a) One statement–reason version:

#	Statement	Reason
1	y is a substring of s	Given
2	v is a substring of y	Given
3	$s = xyz$ for some x, z	(1), def. substring
4	$y = uvw$ for some u, w	(2), def. substring
5	$s = xuvwz$	(3), (4), substitution
6	$s = (xu)\,v\,(wz)$	(5), associativity of concatenation
7	v is a substring of s	(6), def. substring, with $x' = xu$, $z' = wz$

Note how the paragraph's "then $s = xuvwz$" unpacks into three lines (4)–(6). The step the paragraph hides is associativity: the definition of substring needs s written as (string) $\cdot\, v \,\cdot$ (string), and $xuvwz$ only has that shape after you bracket it.

(b) One paragraph version:

> Let v be a suffix of w and let y be a prefix of v. By the definition of suffix there is a string x with $xv = w$, and by the definition of prefix there is a string z with $yz = v$. Substituting the second equation into the first gives $x(yz) = w$, that is, $xyz = w$. Hence $w = (xy)z$, so xy is a prefix of w; and since $xy = x\,y$, the string y is a suffix of xy. Thus y — an arbitrary prefix of a suffix of w — is a suffix of a prefix of w. $\square$

What to check in your own answer: does it start from the givens (not from the conclusion)? Does each equality name the definition used *and* the substitution made? Does the final sentence actually match the definition of the word being concluded ("suffix of a prefix"), rather than just asserting it?

6.2 Problem 7 · Prove it yourself: $L^* = \{\varepsilon\} \cup LL^*$

Let L be **any** language over an alphabet Σ. Prove that

$$L^* = \{\varepsilon\} \cup LL^*.$$

Work through the three steps before writing anything:

- **Requirements.** What does the definition of L^* say? What does LL^* mean? What is the *form* of an element of each side?
- **Intuition.** Test $L = \{\mathtt{a}\}$, then $L = \{\varepsilon\}$, then $L = \emptyset$. Try hard to find an L that breaks the claim.
- **Divide and conquer.** Split into (a) $L^* \subseteq \{\varepsilon\} \cup LL^*$ and (b) $L^* \supseteq \{\varepsilon\} \cup LL^*$. Part (b) splits again into two cases, because its left-hand side is a union.

Write the final answer as a paragraph proof.

Hint (open only once you are properly stuck)

- In (a): take $w \in L^*$. By definition $w = w^{(1)} \cdots w^{(k)}$ with $k \geq 0$ and each $w^{(i)} \in L$. Split on whether $k = 0$ or $k > 0$.
- In (a), case $k > 0$: you need $w = uv$ with $u \in L$ and $v \in L^*$. Which piece is u? Why is the remaining piece in L^*?
- In (b): $w \in \{\varepsilon\} \cup LL^*$ means $w = \varepsilon$ or $w \in LL^*$. Handle the two separately.
- In (b), second case: unfold **both** definitions — $w = uv$ with $u \in L$, $v \in L^*$, and $v = v^{(1)} \cdots v^{(k)}$. Now write w as a product of how many elements of L?

Solution

> To show that $L^* = \{\varepsilon\} \cup LL^*$, we show both inclusions.
>
> ($\subseteq$) Let $w \in L^*$. By the definition of Kleene star, $w = w_1 \cdots w_k$ for some $k \geq 0$ where $w_i \in L$ for each $i = 1, \ldots, k$. We consider two cases. - If $k = 0$, then w is the empty product, so $w = \varepsilon$, and therefore $w \in \{\varepsilon\}$. - If $k > 0$, let $u = w_1$ and $v = w_2 \cdots w_k$. Then $u \in L$, and $v \in L^*$ (it is a product of $k - 1$ strings of L, and the definition of L^*

allows any $k' \geq 0$, including $k' = k-1$). Since $w = uv$ with $u \in L$ and $v \in L^*$, we have $w \in LL^*$.

In both cases $w \in \{\varepsilon\} \cup LL^*$.

($\supseteq$) Let $w \in \{\varepsilon\} \cup LL^*$. Then $w = \varepsilon$ or $w \in LL^*$. - If $w = \varepsilon$, then $w \in L^*$, because ε is the product of $k = 0$ strings of L, which the definition of L^* permits. - If $w \in LL^*$, then by the definition of concatenation $w = uv$ with $u \in L$ and $v \in L^*$; and by the definition of Kleene star, $v = v_1 \cdots v_k$ for some $k \geq 0$ with $v_i \in L$ for each i. Hence $w = uv_1 \cdots v_k$, a product of $k+1$ strings all of which lie in L, so $w \in L^*$.

In both cases $w \in L^*$. Since both inclusions hold, $L^* = \{\varepsilon\} \cup LL^*$. $\square$

Where this usually goes wrong. - Writing "if $k = 0$ then $w = \varepsilon$, obviously" — acceptable later in the module, but right now say *why*: $k = 0$ is the empty product, and the definition allows it. - Forgetting the $k = 0$ case in ($\subseteq$). Then $\varepsilon \in L^*$ never gets placed in $\{\varepsilon\} \cup LL^*$, and the proof has a hole precisely at $L = \emptyset$. - Proving only ($\subseteq$). Set equality needs both directions. - Proving it for $L = \{\mathtt{a}\}$ and stopping there. One example is not a proof of a "for all languages L" statement.

Extra. Prove, in the same style, that $(L^*)^* = L^*$. It is two inclusions again, and the ($\subseteq$) direction is where you must decide what an element of $(L^*)^*$ looks like.

---

7 Part D — Finite, cofinite, and deciding membership

This last problem is about *representation*: a language is an abstract set, but to decide membership on a machine you have to store it somehow. Let $\Sigma = \{\mathtt{a}, \mathtt{b}\}$. Define *FINITE* to be the set of all finite languages over Σ, and

$$\mathit{coFINITE} = \{L \subseteq \Sigma^* \mid \overline{L} \in \mathit{FINITE}\}, \qquad \overline{L} = \Sigma^* \setminus L.$$

Read that definition carefully: *coFINITE* is **not** the same thing as *FINITE*. For example, $\Sigma^* \in$ *coFINITE*, because its complement is $\emptyset$, which is finite.

You may assume: the union of two finite sets is finite.

7.1 Problem 8 · Finite and cofinite

1. If $L \in \mathit{FINITE}$, what data structure could you use to represent L, and given a string w, how would you decide whether $w \in L$?
2. If $L \in \mathit{coFINITE}$, what data structure could you use to represent L, and given a string w, how would you decide whether $w \in L$?
3. Are there any languages in $\mathit{FINITE} \cap \mathit{coFINITE}$? Prove your answer.
4. Are there any languages over Σ that are **not** in $\mathit{FINITE} \cup \mathit{coFINITE}$? Prove your answer.

Solution

1. Store the strings of L in a list (an array, or better a hash set). Since L is finite, the list has some fixed length n. To decide whether $w \in L$: scan the list and compare each entry with w (or look w up in the hash set); accept if a match is found, reject after the list is exhausted. The procedure always terminates because n is finite.

2. Now L itself may be infinite, so store the complement instead: $\overline{L}$ is finite, so keep a list of its strings. To decide whether $w \in L$: scan the list of $\overline{L}$; if w is found, reject (it is in the complement), otherwise accept. So a cofinite language is decided by a *finite* description as well — just a description of what it excludes rather than what it contains.

3. **No** — $\mathit{FINITE} \cap \mathit{coFINITE} = \emptyset$. > Suppose, for contradiction, that $L \in \mathit{FINITE} \cap \mathit{coFINITE}$. Then L is finite and $\overline{L}$ is finite. Since $L \subseteq \Sigma^*$, we have $L \cup \overline{L} = \Sigma^*$; but the union of two finite sets is finite, so Σ^* would be finite. This is false: $\mathtt{a}^n \in \Sigma^*$ for every $n \geq 0$, and these strings are pairwise distinct because they have pairwise distinct lengths, so Σ^* is infinite. Contradiction. Hence no language belongs to both.

   The key move is that the two conditions are about a set and its complement, and together they would force the *universe* Σ^* to be finite.

4. **Yes** — take $L = \{w \in \{\mathtt{a}, \mathtt{b}\}^* \mid |w| \text{ is even}\}$, the language you described in Problem 5.

   - $L \notin \mathit{FINITE}$: for every $k \geq 0$, the string $\mathtt{a}^{2k}$ has even length and so lies in L, and these strings are pairwise distinct. So L is infinite.
   - $L \notin \mathit{coFINITE}$: $\overline{L} = \{w \mid |w| \text{ is odd}\}$, and $\mathtt{a}^{2k+1} \in \overline{L}$ for every $k \geq 0$, again pairwise distinct, so $\overline{L}$ is infinite and hence not finite. > Therefore $L \notin \mathit{FINITE} \cup \mathit{coFINITE}$.

   Other valid witnesses: $\{\mathtt{a}^n\mathtt{b}^n \mid n \geq 0\}$, or $\{w \mid w \text{ contains at least one } \mathtt{a}\}$ (its complement is $\{\mathtt{b}^n \mid n \geq 0\}$, which is infinite). Any language that is infinite *and* has an infinite complement works.

   Why this matters: parts 1 and 2 show that finite and cofinite languages can both be *decided* from a finite amount of stored data. Part 4 shows that most languages are neither, so this trick does not generalise — and deciding membership for the language in part 4 needs a different idea entirely. That is the question the rest of the module is about.

7.2 Optional: check your computations in Python

We represent ε by the empty string `''`, so `set()` is $\emptyset$ and `{''}` is $\{\varepsilon\}$. Run the cell and compare the output with your answers to Problems 4 and 5 — in particular, does `star(set())` agree with your answer to Problem 4.3?

[1]:
```python
# A o B = { xy | x in A, y in B }
def concat(A, B):
    return {x + y for x in A for y in B}


# finite approximation of A*: products of 0 .. max_factors elements of A
def star(A, max_factors=4):
    out, frontier = {""}, {""}
    for _ in range(max_factors):
        frontier = concat(frontier, A)
        out |= frontier
    return out
```

```
# L^R = { w^R | w in L }
def rev(A):
    return {w[::-1] for w in A}


A = {"c", "d"}
B = {"at", "og"}
L = {"aa", "bb"}

print("A o B                =", sorted(concat(A, B)))                        #␣
  ↪Problem 5.1
print("(A o B)^R            =", sorted(rev(concat(A, B))))                   #␣
  ↪Problem 5.3
print("strings of L* len<=4 =", sorted(w for w in star(L) if len(w) <= 4))   #␣
  ↪Problem 5.2
print("empty o {a}          =", concat(set(), {"a"}))                        #␣
  ↪Problem 4.1
print("{eps} o {a}          =", concat({""}, {"a"}))                         #␣
  ↪Problem 4.2
print("empty*               =", star(set()))                                 #␣
  ↪Problem 4.3
print("{eps}*               =", star({""}))                                  #␣
  ↪Problem 4.4
print("is 'a' in L*?        =", "a" in star(L), "  is 'aaaa' in L*? =", "aaaa"␣
  ↪in star(L))
```

```
A o B                = ['cat', 'cog', 'dat', 'dog']
(A o B)^R            = ['goc', 'god', 'tac', 'tad']
strings of L* len<=4 = ['', 'aa', 'aaaa', 'aabb', 'bb', 'bbaa', 'bbbb']
empty o {a}          = set()
{eps} o {a}          = {'a'}
empty*               = {''}
{eps}*               = {''}
is 'a' in L*?        = False   is 'aaaa' in L*? = True
```

7.3 What to take away

- ε, $\emptyset$ and $\{\varepsilon\}$ are three different objects, and computing with them correctly is a matter of unfolding definitions, not of intuition.
- "Prove $X = Y$" means two inclusions. "For all L" is not proved by examples. A proof reads forwards, from the givens to the conclusion.
- A language is a set, but deciding membership needs a *representation*. Finite and cofinite languages have finite representations; Problem 8.4 shows that most languages do not.

The four mistakes to unlearn this week

1. Treating ε as a symbol (it is a string), or as a member of every language (it is a member of every L^*, which is different).
2. Thinking $\emptyset^* = \emptyset$. It is $\{\varepsilon\}$.
3. Giving three examples and calling it a proof of a "for all" statement.
4. Writing a proof that starts at the conclusion and walks backwards to the givens.

Problems adapted from CSE 30151, Theory of Computing, University of Notre Dame.